

Chat With Me!

Кроссплатформенное мобильное чат-приложение на Flutter

Flutter Developer (08.2025)



Тема: Чат с использованием FireStore



Джалилов Ридваан

Фронтенд разработчик



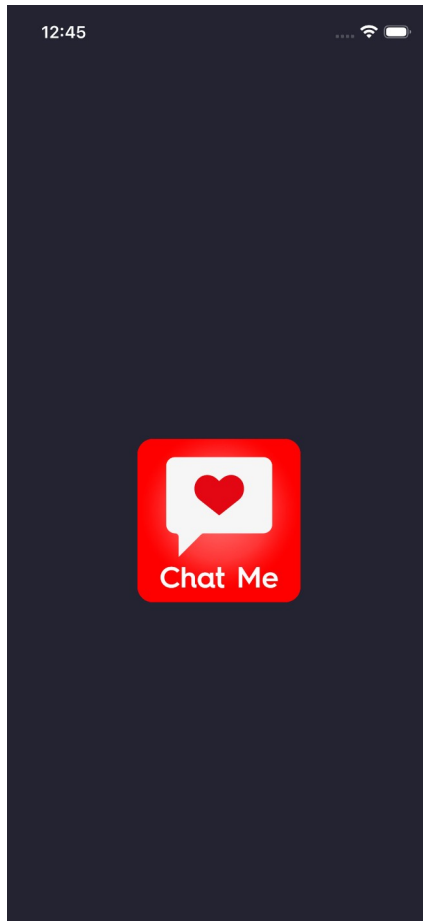
<https://github.com/rddeveloper2019/>





Цель:

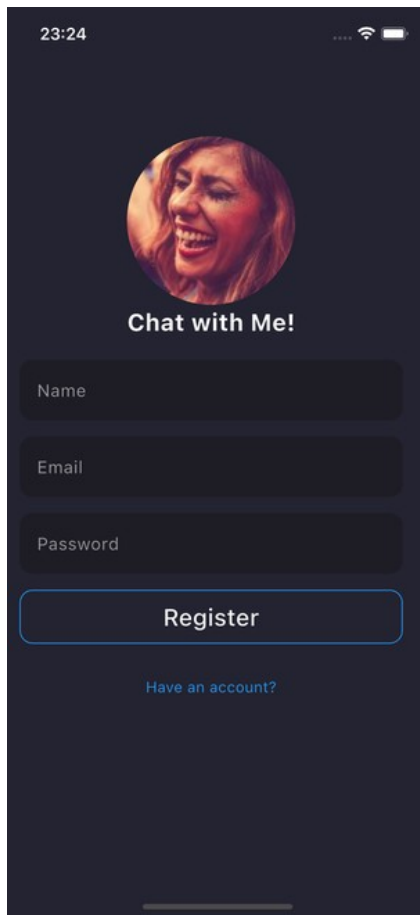
Создать современное, безопасное и масштабируемое мобильное приложение для обмена сообщениями с поддержкой личных и групповых чатов.





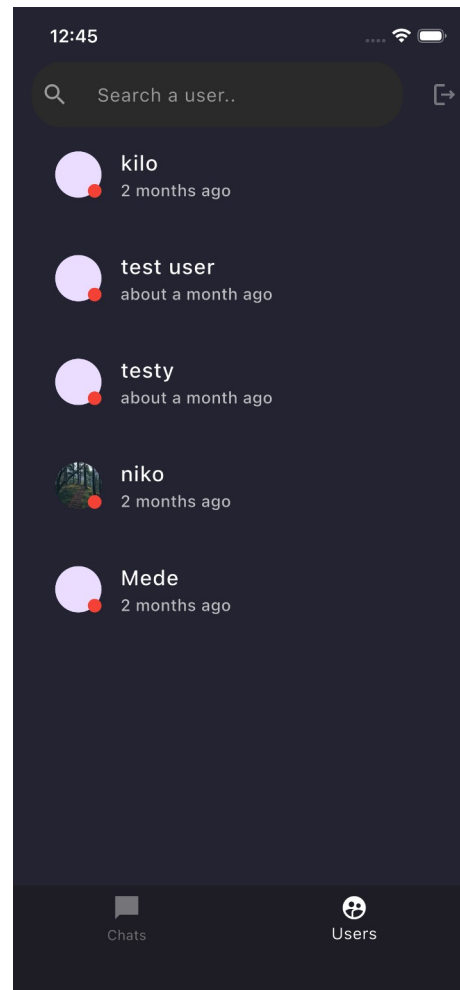
Задачи:

- Реализовать авторизацию и регистрацию пользователей через Email
- Обеспечить обмен текстовыми сообщениями и изображениями в реальном времени
- Поддержать личные и групповые чаты с неограниченным числом участников
- Реализовать поиск пользователей и отображение статуса «онлайн/офлайн»
- Обеспечить безопасное хранение данных через Firebase
- Спроектировать чистую и поддерживаемую архитектуру приложения



Ключевой функционал

 Авторизация	Регистрация/вход через Firebase Auth
 Чаты	Личные и групповые, реальное время
 Медиа	Отправка изображений с предпросмотром
 Поиск	Поиск пользователей по имени
 Статусы	Индикатор «последний раз в сети»
 Управление	Удаление чатов с очисткой данных
 UX	Авто-разфокусировка полей ввода



Структура проекта (Clean Architecture)

lib/

models/	# Data models (User, Chat, Message)
services/	# Firebase services (Auth, Firestore, Storage)
providers/	# State management (Provider + ChangeNotifier)
screens/	# UI screens (Login, Chat, Search, etc.)
widgets/	# Reusable UI components
utils/	# Helpers, constants, themes

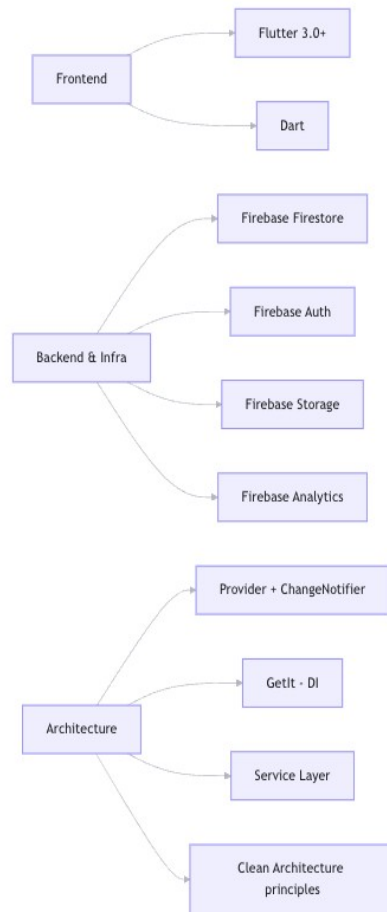
Архитектура проекта обеспечивает:

Чистое разделение ответственности – каждый слой знает только о своих задачах

Тестируемость – можно мокать любой слой

Масштабируемость – легко добавлять новые фичи

Поддерживаемость – код легко читать и изменять



```
lib/  
├── main.dart                # Точка входа, инициализация Firebase, DI  
├── models/                 # Модели данных (User, Chat, Message)  
│   ├── user_model.dart  
│   ├── chat_model.dart  
│   └── message_model.dart  
├── providers/              # Слой состояния (Provider + ChangeNotifier)  
│   ├── auth_provider.dart  
│   ├── chat_provider.dart  
│   └── user_search_provider.dart  
├── services/               # Бизнес-логика и работа с Firebase  
│   ├── auth_service.dart  
│   ├── firestore_service.dart  
│   ├── storage_service.dart  
│   └── chat_service.dart  
├── pages/                  # UI-экраны  
│   ├── login_page.dart  
│   ├── register_page.dart  
│   ├── chat_list_page.dart  
│   ├── chat_page.dart  
│   └── user_search_page.dart  
└── widgets/                # Переиспользуемые компоненты  
    ├── message_bubble.dart  
    ├── chat_tile.dart  
    └── image_preview.dart
```



Основные пакеты:

provider: ^6.0.0

get_it: ^7.0.0

cloud_firestore: ^4.0.0

firebase_auth: ^4.0.0

firebase_storage: ^11.0.0

flutter_keyboard_visibility

Управление состоянием

Внедрение зависимостей

Работа с БД

Аутентификация

Хранение файлов

Управление клавиатурой



✓ Достигнуто:

- Рабочий прототип чат-приложения с полным циклом: от авторизации до отправки медиа
 - Интеграция с Firebase-экосистемой для бэкенда «без сервера»
- Применение паттернов Provider + GetIt для управляемого состояния и тестирования
- Кроссплатформенность: сборка под iOS, Android, Web, Desktop

💡 Извлечённые уроки:

Важность продуманной структуры данных в Firestore для избежания дублирования

Необходимость единого подхода к обработке состояний загрузки и ошибок

Потенциал перехода на BLoC для сложной бизнес-логики и лучшей тестируемости

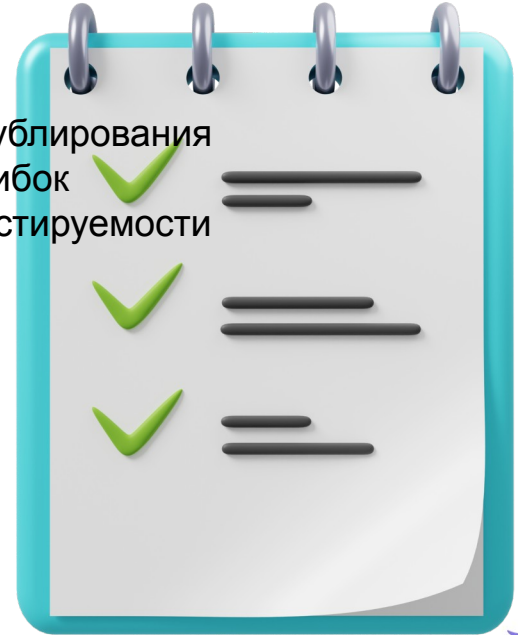
🚀 Следующие шаги:

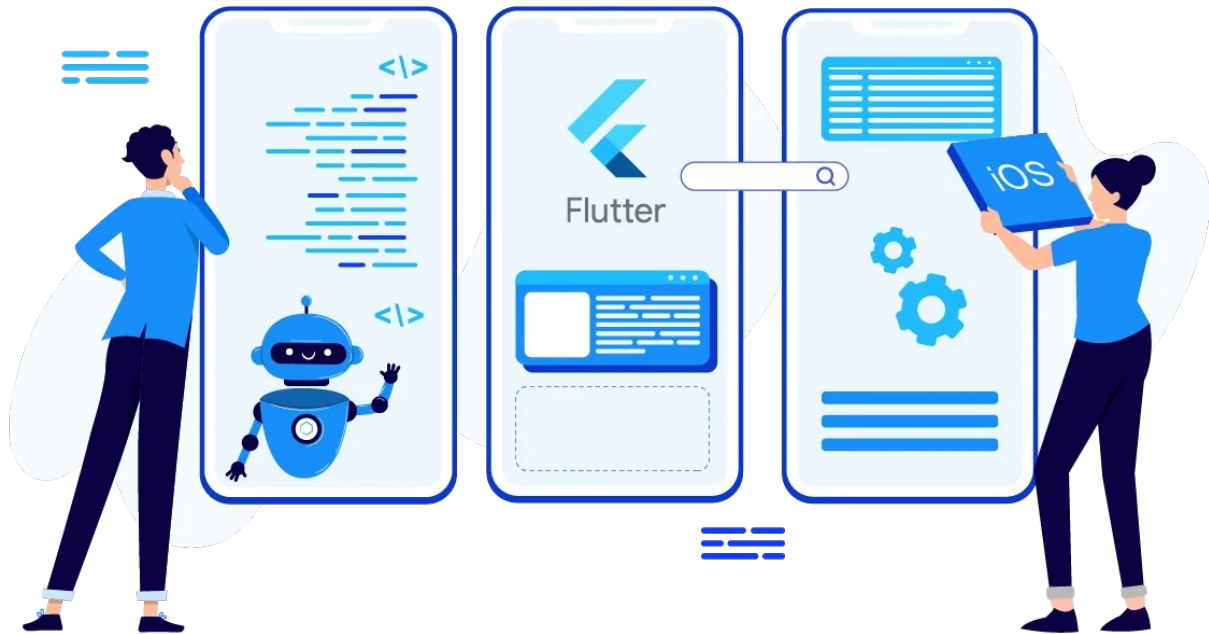
Рефакторинг архитектуры (Clean Architecture + BLoC + multi-module)

Добавление push-уведомлений через Firebase Cloud Messaging

Покрытие ключевых сценариев юнит- и интеграционными тестами

Подготовка к публикации в магазины приложений





https://github.com/rddeveloper2019/chat_with_me



<https://otus.ru/lessons/dart-flutter/>